CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi
Lecturer, Department of Computer Science and Engineering
United International University

MIPS Code for Conditional Branch

Instruction	Operation	Description
`beq $x, $y, LABEL`	If `$x == $y`, jump to `LABEL`	Branches if `$x` and `$y` are equal.
`bne $x, $y, LABEL`	If `$x != $y`, jump to `LABEL`	Branches if `$x` and `$y` are **not** equal.
`slt $t, $x, $y`	`$t = 1` if `$x < $y`, else `$t = 0`	Sets `$t` to 1 if `$x` is less than `$y`.
`slti $t, $x, constant`	`$t = 1` if `$x < constant`, else `$t = 0`	Compares register `$x` with an **immediate constant** and stores 1 or 0 in `$t`.

MIPS Code for Conditional Branch

Example:
if (i == j) f = g + h;
else f = g –h;

Let, compiler will assign f at $s0, g at $s1, h at $s2, i at $s3 and j at $s4

Assembly Code for the following Jump Code

```
beq  $s3, $s4, IF_PART  # If i == j, branch to IF_PART
ELSE_PART:
   sub  $s0, $s1, $s2  # f = g - h
   j    END_IF         # Jump to end

IF_PART:
   add  $s0, $s1, $s2  # f = g + h
END_IF:
```

MIPS Branching Example

Example:
if (i < j) f = g + h;
else f = g –h;

Let, compiler will assign f at $s0, g at $s1, h at $s2, i at $s3 and j at $s4

MIPS Branching Example Solution

Example:
if (i < j) f = g + h;
else f = g –h;

Let, compiler will assign f at $s0, g at $s1, h at $s2, i at $s3 and j at $s4
Solution:

```
slt  $t0, $s3, $s4
beq  $to, $zero, Else
add $s0, $s1, $s2
j Exit
Else: sub $s0, $s1, $s2
Exit:
```

MIPS Branching Example

Example:
if (i <= j) f = g + h;
else f = g –h;

Let, compiler will assign f at $s0, g at $s1, h at $s2, i at $s3 and j at $s4

MIPS Code for the Previous Example

```
    slt  $t0, $s3, $s4   # $t0 = 1 if i < j, else $t0 = 0
     beq  $t0, $zero, ELSE_PART  # If $t0 == 0 (i >= j), jump to ELSE_PART

IF_PART:
    add  $s0, $s1, $s2   # f = g + h
    j    END_IF          # Jump to END_IF to skip ELSE_PART

ELSE_PART:
    sub  $s0, $s1, $s2   # f = g - h

END_IF:
```

Loop in MIPS

Example:
while(save[5] == k) i++;
Let, compiler will assign I at $s0,k at $s1 and save at $s2

Solution:
Loop:
lw $t0, 20($s2)
bne $t0, $s1, Exit
addi $s0, $s0, 1
j Loop
Exit:

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

```
addi $s2, $zero, 20
addi $s3, $zero, 30
```

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

add $s1, $s2, $s3

MIPS Assembly Code Function Conversion

```
int function int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

g= $a0, h= $a1, i=$a2, j =$a3

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

add $t0, $a0, $a1

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

add $t1, $a2, $a3

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

sub $s0, $t0, $t1

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

add $s0, $s1, $s0

MIPS Assembly Code Function Conversion

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero, $s0
}
```

Arguments will be found at $a0-$a3 Let, compiler will assign f at $s0, a at $s1, b at $s2, c at $s3

Partial Solution:

add $v0, $s0, $zero

Solution for Conversion

Solution (Partial):

```
addi $s2, $zero, 20
addi $s3, $zero, 30
add $s1, $s2, $s3
add $t0, $a0, $a1
add $t1, $a2, $a3
sub $s0, $t0, $t1
add $s0, $s1, $s0
add $v0, $s0, $zero //return f // $v0=f
```

Additional observations

$t0–$t9: 10 temporary registers that are not preserved by the callee (called procedure) on a procedure call.

$s0–$s7: 8 saved registers that must be preserved on a procedure call (if used, the callee saves and restores them)

the calling program, or caller, puts the parameter values in $a0–$a3 and uses jal X to jump to procedure X (sometimes named the callee). The callee then performs the calculations, places the results in $v0–$v1, and returns control to the caller using jr $ra

Saving Registers on the Stack

To save register values before calling a function

- When a function is called, it may use some registers that the caller is already using.
- We save the old values on the stack so we can restore them after the function finishes.

To store local variables for a function

If a function has local variables that don’t fit in registers, they can be stored on the stack.

Stack Pointer

The stack pointer ($sp) helps manage memory on the stack, which is used to store temporary data — like **local variables, return addresses, or saved registers** — especially during function calls.

- **$sp always points to the top of the stack.**
- **The stack grows downward — so when you allocate space, you subtract from $sp.**

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

There are 4 local variables.

Each local variable occupies 4 bytes of data.
So we need total stack allocation of 4 x 4 =16 slots.

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

Allocating 16 slots

addi $sp, $sp, -16

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

Allocating 16 slots

```
addi $sp, $sp, -16
```

```
sw $s0, 12($sp)
```

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

Allocating 16 slots

```
addi $sp, $sp, -16

sw $s0, 12($sp)
sw $s1, 8($sp)
```

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

Allocating 16 slots

```
addi $sp, $sp, -16

sw $s0, 12($sp)
sw $s1, 8($sp)
sw $s2, 4($sp)
```

Demonstration of Stack Pointer

```
int function(int g, int h, int i, int j)
{
int f, a, b = 20, c = 30;
a = b + c;
f = ( g + h ) - (i + j);
f = a + f;
return f; $v0=f, add $v0, $zero,
$s0
}
```

Allocating 16 slots

```
addi $sp, $sp, -16

sw $s0, 12($sp)
sw $s1, 8($sp)
sw $s2, 4($sp)
sw $s3, 0($sp)
```


UIU
UNITED INTERNATIONAL UNIVERSITY
QUEST
FOR
EXCELLENCE

Thank you!